

Pandas Cheat Sheet

Making a DataFrame

```
data = {"col_header1": [col1d1, col1d2, col1d3], "col_header2": [col2d1, col2d2, col2d3]}
df=pd.DataFrame(data)
```

Making a DataFrame with an Index

```
data = {"col_header1": [col1d1, col1d2, col1d3], "col_header2": [col2d1, col2d2, col2d3]}
df=pd.DataFrame(data, index = ['val1', 'val2', 'val3'])
```

Get a Row as a Series

```
my_series = df.loc[0] # replace 0 with the row num you want
```

Get a DF

```
my_df = df.loc[0, 1] # replace values in list with the row nums you want
```

Getting Info about a DataFrame

- df.columns # the col headers
- df.index # the index values
- df.describe() # stat summary of df
- df.info() # description of df

Indexing and Selecting Data

- .loc # label-based
- .iloc # int-based
- Use loc & iloc with [] and a single label/int, list of labels/int, slice of labels/int, pair of slices, or comma-separated lists inside the [] like print(df.iloc[1, 3])

Getting A Scalar With at And iat

- some_val = df.at['label', 'another_label'] # label-based
- some_val = df.iat[an_int, another_int] # int-based

Set Index

```
df.set_index('col_name')
```

Sorting Your DataFrame with sort_values()

- new_df = df.sort_values(by='col_name', ascending=True) # or use inplace = True
- Can give function a list of col names
- Can give function a list of ascending= Booleans

Detecting empty cells

- isna()
- notna()
- notnull()

Dealing with empty cells

- new_df = df.dropna() # or use inplace = True
- new_df = df.fillna(value_to_fill_with) # or use inplace = True

Detecting & removing duplicates

- my_series = df.duplicated()
- new_df = df.drop_duplicates() # or use inplace = True

Renaming columns

```
df.rename(columns = {'old_name':'new_name'}, inplace = True)
```

Making a new column from existing columns

example:

```
df['new_col_name'] = df['existing_col_name'] + 2 * (df['another_existing_col_name'])
```

Adding new columns to an existing DataFrame

```
new_col_data = [data1, data2, data3, ... ]
```

```
df[new_col_name] = new_col_data
```

Deleting rows and columns from a DataFrame

Drop columns

- `df.drop(["col_to_drop_1", "col_to_drop_2"], axis = 1, inplace = True)`

Drop rows

- `df.drop(["index_to_drop_1", "index_to_drop_2"], inplace = True)`

Conditional selection

```
new_df = df[df['column_name'] > 0]    # can replace > 0 with a different condition
```

Concatenating DataFrames

```
new_df = pd.concat([df1, df2])
```

Aggregation

`df.mean()` will get mean of each column of DataFrame `df`

`df.mean(axis = 'columns')` will get mean of each row of DataFrame `df`

Group by 158

```
new_df = df.groupby('col_name').mean()    # or replace mean with another aggregate function
```

```
new_df = df.groupby('col_name')['another_col_name'].mean()    # groups by 'col_name' but does  
aggregate function on 'another_col_name'
```

Can replace 'col_name' in groupby with list to group by multiple columns

Retrieving a column (or columns)

```
some_col = df.col_name
```

```
some_col = df['col_name']
```

```
new_df = df[['col_name1', 'col_name2']]
```

Transposing a DataFrame

```
new_df = df.T
```

Reordering columns

```
new_df = df.loc[:, [list of 'cols' in new order]]
```

```
new_df = df.iloc[:, [list of cols in new order]]
```

Mapping

Return a new series with the values from the old series having been adjusted as per a function

```
df['some_col'] = df['some_col'].map(some_function)
```